Social Graph — Friends & Followers

<details>
<summary>Relevant source files</summary>

- Backend/STUDER/src/main/java/facu/studer/DTOs/messages/ChatListPageResponseDTO.java
- Backend/STUDER/src/main/java/facu/studer/DTOs/messages/DirectMessagePageResponseDTO.java
- Backend/STUDER/src/main/java/facu/studer/DTOs/messages/DirectMessageResponseDTO.java
- Backend/STUDER/src/main/java/facu/studer/DTOs/user/FollowRequestDTO.java
- Backend/STUDER/src/main/java/facu/studer/DTOs/user/FriendResponseDTO.java
- Backend/STUDER/src/main/java/facu/studer/DTOs/user/FriendsListResponseDTO.java
- Backend/STUDER/src/main/java/facu/studer/controllers/DirectMessageController.java
- Backend/STUDER/src/main/java/facu/studer/controllers/FriendController.java
- Backend/STUDER/src/main/java/facu/studer/entities/Friend.java
- Backend/STUDER/src/main/java/facu/studer/services/FriendService.java
- Backend/STUDER/src/main/java/facu/studer/services/implementation/DirectMessageServiceImpl.java
- Backend/STUDER/src/main/java/facu/studer/services/implementation/FriendServiceImpl.java
</details>

The STUDER social graph is built on a non-reciprocal "follow" mechanism that evolves into a reciprocal "friendship" once both parties have followed each other. This model governs access to private features, specifically direct messaging, and drives the notification system for user engagement.

Data Model: The Friend Entity

The social graph is persisted via the `Friend` entity `Backend/STUDER/src/main/java/facu/studer/entities/Friend.java 15-22` Unlike a simple join table, this entity uses a two-flag state model to track the status of the relationship from both perspectives.

Entity Schema

Field	Type	Description
`sender`	`User`	The user who initiated the first follow action `Backend/STUDER/src/main/java/facu/studer/entities/Friend.java 28`
`senderAccept`	`Boolean`	True if the sender is currently following the receiver `Backend/STUDER/src/main/java/facu/studer/entities/Friend.java 31`
`receiver`	`User`	The target of the follow action `Backend/STUDER/src/main/java/facu/studer/entities/Friend.java 38`
`receiverAccept`	`Boolean`	True if the receiver has followed the sender back `Backend/STUDER/src/main/java/facu/studer/entities/Friend.java 41`

Logic State Transitions

A "friendship" (confirmed status) only exists when `senderAccept == true` AND `receiverAccept == true`

`Backend/STUDER/src/main/java/facu/studer/services/implementation/FriendServiceImpl.java 147-157`

Sources: `Backend/STUDER/src/main/java/facu/studer/entities/Friend.java 1-43`

`Backend/STUDER/src/main/java/facu/studer/services/implementation/FriendServiceImpl.java 147-157`

Implementation: FriendServiceImpl

The `FriendServiceImpl` handles the business logic for creating and updating relationships. It is designed to be idempotent and handles the transition from "follower" to "friend" automatically.

The Follow Flow

When a user attempts to follow another, the system performs the following checks:

1. **Self-follow Check**: Users cannot follow themselves
   `Backend/STUDER/src/main/java/facu/studer/services/implementation/FriendServiceImpl.java 72-74`
2. **Reverse Request Check**: If the target user had already followed the current user, the existing record is updated to `receiverAccept = true`, completing the friendship

`Backend/STUDER/src/main/java/facu/studer/services/implementation/FriendServiceImpl.java` 82-86

3. **New Request**: If no record exists, a new `Friend` entity is created with the current user as `sender` and `senderAccept = true`

   `Backend/STUDER/src/main/java/facu/studer/services/implementation/FriendServiceImpl.java` 94-103

Notification Trigger

Every successful follow action (either a new follow or completing a friendship) triggers a `USER` type notification for the target user

`Backend/STUDER/src/main/java/facu/studer/services/implementation/FriendServiceImpl.java` 111-113

Friendship Logic Diagram

The following diagram maps the natural language logic of following to the code implementation in `FriendServiceImpl`.

"Friendship State Logic"

Sources:

`Backend/STUDER/src/main/java/facu/studer/services/implementation/FriendServiceImpl.java` 63-116

`Backend/STUDER/src/main/java/facu/studer/controllers/FriendController.java` 38-57

REST API Endpoints

The `FriendController` exposes endpoints for managing the social graph. All endpoints are protected and require a valid JWT

`Backend/STUDER/src/main/java/facu/studer/controllers/FriendController.java` 19-21

Method	Endpoint	Description
`POST`	`/api/v1/friends/follow`	Follow a user via `FollowRequestDTO` body `Backend/STUDER/src/main/java/facu/studer/controllers/FriendController.java` 38-44

Method	Endpoint	Description
`POST`	`/api/v1/friends/follow/{userId}`	Follow a user via path variable `Backend/STUDER/src/main/java/facu/studer/controllers/FriendController.java` `52-57`
`DELETE`	`/api/v1/friends/follow/{userId}`	Unfollow a user (sets both flags to `false`) `Backend/STUDER/src/main/java/facu/studer/controllers/FriendController.java` `66-75`
`GET`	`/api/v1/friends`	List confirmed friends (paginated) `Backend/STUDER/src/main/java/facu/studer/controllers/FriendController.java` `84-90`
`GET`	`/api/v1/friends/status/{userId}`	Get relationship status with a specific user `Backend/STUDER/src/main/java/facu/studer/controllers/FriendController.java` `98-103`

Sources: `Backend/STUDER/src/main/java/facu/studer/controllers/FriendController.java` `1-104`

Social Graph and Messaging Integration

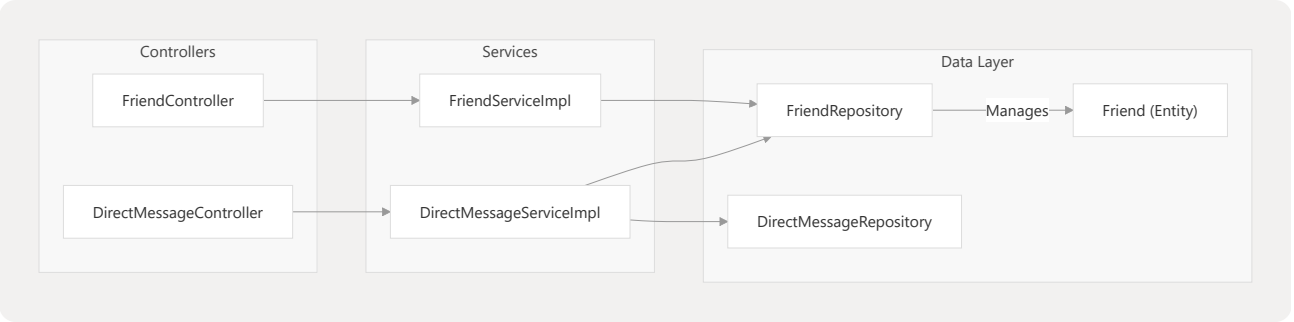
The social graph serves as a gatekeeper for the messaging system. The `DirectMessageServiceImpl` enforces friendship constraints for various operations.

1. **Chat Access**: Users can only retrieve a full chat history (`getChatById`) if a confirmed friendship exists
   `Backend/STUDER/src/main/java/facu/studer/services/implementation/DirectMessageServiceImpl.java` `79-82`
2. **Chat List**: The list of active conversations (`getAllChats`) is filtered to only include users with confirmed friendship status
   `Backend/STUDER/src/main/java/facu/studer/services/implementation/DirectMessageServiceImpl.java` `114-117`

Code Entity Association Diagram

This diagram bridges the REST controllers to the underlying Service and Repository entities.

"Messaging and Social Graph Integration"



Sources:

Backend/STUDER/src/main/java/facu/studer/services/implementation/DirectMessageServiceImpl.java	79-82
Backend/STUDER/src/main/java/facu/studer/services/implementation/FriendServiceImpl.java	37-51
Backend/STUDER/src/main/java/facu/studer/controllers/DirectMessageController.java	1-109